UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

TRABAJO PRÁCTICO INTEGRADOR

PROGRAMACIÓN I

Sistema de Control de Inventario para una ferretería

Alumna: Lorena Mercedes Orieta
Materia: Programación I
Carrera: Tecnicatura Universitaria en Programación
Fecha: noviembre de 2026

Informe de diseño, fundamentación y pruebas
Archivo fuente: ferreteria_nuevo.py

Índice

1. Presentación del problema y objetivos

2. Requisitos y alcance del sistema

3. Diseño de datos y organización del código

4. Fundamentación de los conceptos utilizados

5. Funcionamiento de las siete opciones

6. Estrategia de validación y manejo de errores

7. Lenguaje, entorno y recursos de desarrollo

8. Diagramas de flujo de la solución

9. Pruebas de funcionamiento

   9.1. Casos positivos observados en la ejecución de la aplicación

   9.2. Errores y situaciones límite revisadas

   9.3. Evidencias de ejecución en Visual Studio Code

10. Resultados, límites y conclusiones

11. Fuentes consultadas

1. Presentación del problema y objetivos

Una ferretería que registra manualmente sus productos puede tener dificultades para conocer cuántas herramientas quedan disponibles después de una venta o de un ingreso de mercadería. Para resolver este problema se desarrolló una aplicación de consola en Python que permite mantener actualizado un inventario mientras el programa está en ejecución.

El sistema no pretende reemplazar a una aplicación comercial: su propósito es aplicar los contenidos de Programación I mediante una solución funcional, organizada en funciones y preparada para responder correctamente ante errores de ingreso de datos.

1.1. Objetivo general

Diseñar e implementar un sistema interactivo que registre herramientas y actualice sus existencias mediante operaciones de carga, consulta, alta, venta e ingreso de mercadería.

1.2. Objetivos específicos

- Representar las herramientas mediante una lista de diccionarios con las claves indicadas por la cátedra.
- Organizar el programa en funciones independientes y un menú que permanezca activo hasta elegir salir.
- Evitar nombres vacíos, productos duplicados, cantidades no enteras y movimientos que produzcan stock negativo.

- Mostrar información comprensible para el usuario, tanto en operaciones correctas como en situaciones de error.
- Documentar el diseño y comprobar el funcionamiento mediante casos de prueba.

2. Requisitos y alcance del sistema

El programa muestra siete opciones y vuelve al menú después de cada operación, salvo al seleccionar la opción 7. La actualización del inventario es inmediata durante la ejecución: una venta descuenta unidades y un ingreso las suma. No existe almacenamiento permanente; al cerrar el programa se pierden los datos ingresados.

Opción	Acción	Resultado principal
1	Carga múltiple	Registra varias herramientas con stock inicial válido.
2	Mostrar inventario	Lista las herramientas y su stock actual.
3	Consulta de stock	Busca por nombre e informa unidades disponibles.
4	Reporte de agotados	Muestra productos con cantidad igual a cero.
5	Alta individual	Agrega una herramienta nueva sin duplicados.
6	Movimiento de stock	Resta por venta o suma por ingreso.
7	Salir	Finaliza el ciclo del menú.

El sistema utiliza únicamente las estructuras habilitadas por el enunciado. No emplea clases, bases de datos, archivos para guardar el inventario ni colecciones adicionales. La ausencia de persistencia es deliberada y responde al alcance establecido en la consigna.

3. Diseño de datos y organización del código

3.1. Estructura de datos principal

El inventario se declara dentro de main() como una lista vacía, denominada inventario. Cada elemento es un diccionario con exactamente dos claves: herramienta (nombre) y cantidad (stock entero disponible).

```
inventario = []
# Ejemplo luego de la carga:
[{'herramienta': 'Martillo', 'cantidad': 10},
 {'herramienta': 'Pala', 'cantidad': 0}]
```

Esta representación permite guardar varias herramientas dentro de una sola estructura y recorrerlas con for. Para incorporar una herramienta se utiliza inventario.append({...}), creando directamente el diccionario en la lista. No se construyen diccionarios sueltos ni otras listas auxiliares.

3.2. Funciones y responsabilidades

Función	Responsabilidad
cargar_herramientas(inventario)	Solicitar y validar carga múltiple; reintentar datos incorrectos.
mostrar_inventario(inventario)	Mostrar todos los registros o avisar si está vacío.
consultar_stock(inventario)	Buscar un nombre y mostrar su cantidad.
reporte_agotados(inventario)	Recorrer la lista y mostrar solo stock cero.
alta_producto(inventario)	Incorporar un producto individual si sus datos son válidos.
actualizar_stock(inventario)	Controlar ventas e ingresos sin alterar stock ante errores.
main()	Crear inventario, mostrar menú, coordinar operaciones y capturar errores.

Todas las funciones que necesitan el inventario lo reciben como parámetro. Por lo tanto, no existen variables globales. Como la lista es mutable, una actualización realizada por una función queda reflejada en el mismo inventario de main(), sin necesidad de devolver una copia.

4. Fundamentación de los conceptos utilizados

4.1. Listas y diccionarios

La lista permite almacenar una cantidad variable de herramientas y recorrerlas en el orden en que fueron registradas. Cada diccionario representa una herramienta y relaciona sus dos atributos mediante las claves herramienta y cantidad. Por ejemplo, al acceder a producto["cantidad"] se obtiene el stock de ese producto. Esta combinación es apropiada porque todos los registros comparten el mismo formato.

4.2. Funciones

Las funciones separan las tareas y evitan escribir todo el programa dentro de main(). Si debe modificarse la consulta de stock, el cambio se concentra en consultar_stock(). Esta modularización también facilita las pruebas: cada opción tiene una responsabilidad clara y recibe el inventario como argumento.

4.3. Condicionales

Las estructuras if, elif y else permiten decidir qué opción ejecutar, verificar que un nombre sea válido o determinar si una venta dispone de existencias suficientes. En actualizar_stock(), por ejemplo, una venta solo se realiza si las unidades solicitadas no superan el stock disponible.

4.4. Bucles

while True mantiene visible el menú hasta que el usuario elige salir. La carga múltiple utiliza for para repetir el registro según la cantidad elegida, y otros while permiten volver a pedir un dato

inválido sin reiniciar toda la operación. Además, for recorre inventario para localizar herramientas, listar productos o detectar agotados.

4.5. Manejo de excepciones: try/except y raise

int() produce ValueError cuando una entrada no es un número entero válido, como abc o 3.5. Por su parte, raise ValueError permite informar reglas de negocio incumplidas: cantidad negativa, nombre vacío, producto repetido o venta superior al stock. La combinación con try/except evita que el programa se interrumpa de forma inesperada ante esos errores.

```
if unidades > producto_encontrado["cantidad"]:
    raise ValueError("No hay stock suficiente para realizar la venta.")
```

Se establecen dos comportamientos: en la opción 1, el error se captura dentro de la misma función y se vuelve a pedir el dato; en las opciones 3, 5 y 6, ciertas excepciones llegan al except de main(), que informa el problema y vuelve al menú. Esta diferencia responde a la consigna.

4.6. Normalización de nombres y variables locales

Los nombres se procesan con strip() para quitar espacios exteriores y se comparan con lower() para ignorar diferencias entre mayúsculas y minúsculas. Así, Martillo, martillo y MARTILLO se reconocen como un mismo producto. El nombre se guarda en inventario sin los espacios del inicio y del final, respetando las mayúsculas que escribió el usuario; la comparación con lower() se usa solo para detectar duplicados y para buscar herramientas. Los valores se almacenan en variables locales y el inventario se comparte por parámetros.

5. Funcionamiento de las siete opciones

5.1. Opción 1 - Carga múltiple

Primero pide cuántas herramientas distintas se cargarán (entero mayor que cero). Para cada herramienta solicita un nombre no vacío ni duplicado y después un stock entero mayor o igual a cero. Si falla el nombre vuelve a pedir solo el nombre; si falla el stock conserva el nombre correcto. El registro se agrega únicamente cuando ambos campos son válidos. Si la opción se ejecuta más de una vez, las nuevas herramientas se suman a las ya cargadas y el control de duplicados considera todo el inventario.

5.2. Opción 2 - Visualización

Si la lista está vacía informa esa situación. De lo contrario, recorre cada diccionario y muestra el nombre de la herramienta y la cantidad disponible.

5.3. Opción 3 - Consulta

Comprueba primero que existan herramientas. Solicita el nombre, rechaza el vacío mediante ValueError y recorre inventario para buscar coincidencias sin distinguir mayúsculas. Si no existe, lo informa sin lanzar una excepción.

5.4. Opción 4 - Agotados

Muestra solo productos cuya cantidad es cero. Utiliza un contador simple para distinguir entre inventario vacío, existencia de agotados y catálogo sin agotados. No crea listas adicionales.

5.5. Opción 5 - Alta individual

Solicita el nombre de una herramienta nueva y verifica que no esté vacío ni repetido. Recién entonces pide el stock inicial. Si un dato es inválido, la operación termina sin agregar nada y el error llega al menú principal.

5.6. Opción 6 - Actualización

Comprueba existencia del inventario, validez del nombre y existencia de la herramienta. Acepta solo los movimientos 1 (venta) y 2 (ingreso), con unidades enteras positivas. Para ventas verifica que haya existencias suficientes antes de restar; para ingresos suma unidades. En cualquier error deja el stock sin modificaciones. Un movimiento distinto de 1 o 2 se informa con un mensaje, mientras que las unidades inválidas se rechazan mediante ValueError.

5.7. Opción 7 - Salida

Finaliza el ciclo while de main() con break después de mostrar un mensaje de despedida. Al no guardar datos en archivo ni base de datos, el inventario se pierde al cerrar el proceso.

6. Estrategia de validación y manejo de errores

Para evitar registros incompletos, las modificaciones se realizan solo después de comprobar las condiciones necesarias. En la carga múltiple, un error obliga a reintentar el campo correspondiente; en el alta individual y la actualización, el error devuelve al menú sin cambiar datos.

Situación	Mecanismo aplicado	Consecuencia
Menú: abc o 3.5	int() + except ValueError	Mostrar error y pedir nuevamente opción.
Menú: 0 o 9	raise ValueError	Volver al menú.
Carga: número ≤ 0	raise + except local	Repetir cantidad a cargar.
Nombre vacío o duplicado	raise ValueError	Repetir nombre (opción 1) o volver al menú (opción 5).
Stock inicial negativo	raise ValueError	Repetir stock (opción 1) o volver al menú (opción 5).
Stock no entero	int() + except	No agregar registros incompletos.
Consulta de nombre inexistente	Condición + mensaje	Informar sin excepción.
Movimiento distinto de 1 o 2	Condición + mensaje	Volver sin modificar stock.
Unidades ≤ 0	raise ValueError	No modificar stock.

Venta mayor al stock	raise ValueError	Conservar existencias previas.

Criterio: *Un stock inicial de 0 es válido; una venta o ingreso de 0 unidades no lo es. Esta diferencia se mantiene deliberadamente en el programa.*

7. Lenguaje, entorno y recursos de desarrollo

7.1. Recursos utilizados

Recurso	Descripción y motivo de elección
Python 3	Lenguaje interpretado de propósito general. Se eligió porque permite aplicar listas, diccionarios, funciones, condicionales, bucles y excepciones con una sintaxis accesible. Versión instalada: 3.14.5.
Visual Studio Code	Editor de código utilizado para escribir, guardar y ejecutar ferreteria_nuevo.py. Permite trabajar con archivos .py y utilizar la terminal integrada para las pruebas. Versión instalada: 1.145.2.
Terminal integrada	Interfaz de texto desde la que se presenta el menú, se reciben datos con input() y se muestran resultados con print().
Windows	Sistema operativo anfitrión del equipo utilizado para editar y ejecutar el programa. Edición y versión: Windows 11, versión 25H2.
Biblioteca estándar / funciones incorporadas	Se utilizan input(), print(), int(), len(), range(), append(), strip(), lower(), f-strings para armar los mensajes y la excepción ValueError. El programa no depende de paquetes externos.

7.2. Procedimiento de ejecución

1. Abrir la carpeta del proyecto en Visual Studio Code.
2. Abrir el archivo ferreteria_nuevo.py.
3. Ejecutar el archivo con el botón de Python o desde la terminal con el comando python ferreteria_nuevo.py.
4. Seleccionar una opción de 1 a 7 y responder las solicitudes que aparecen en pantalla.
5. Para finalizar, elegir la opción 7.

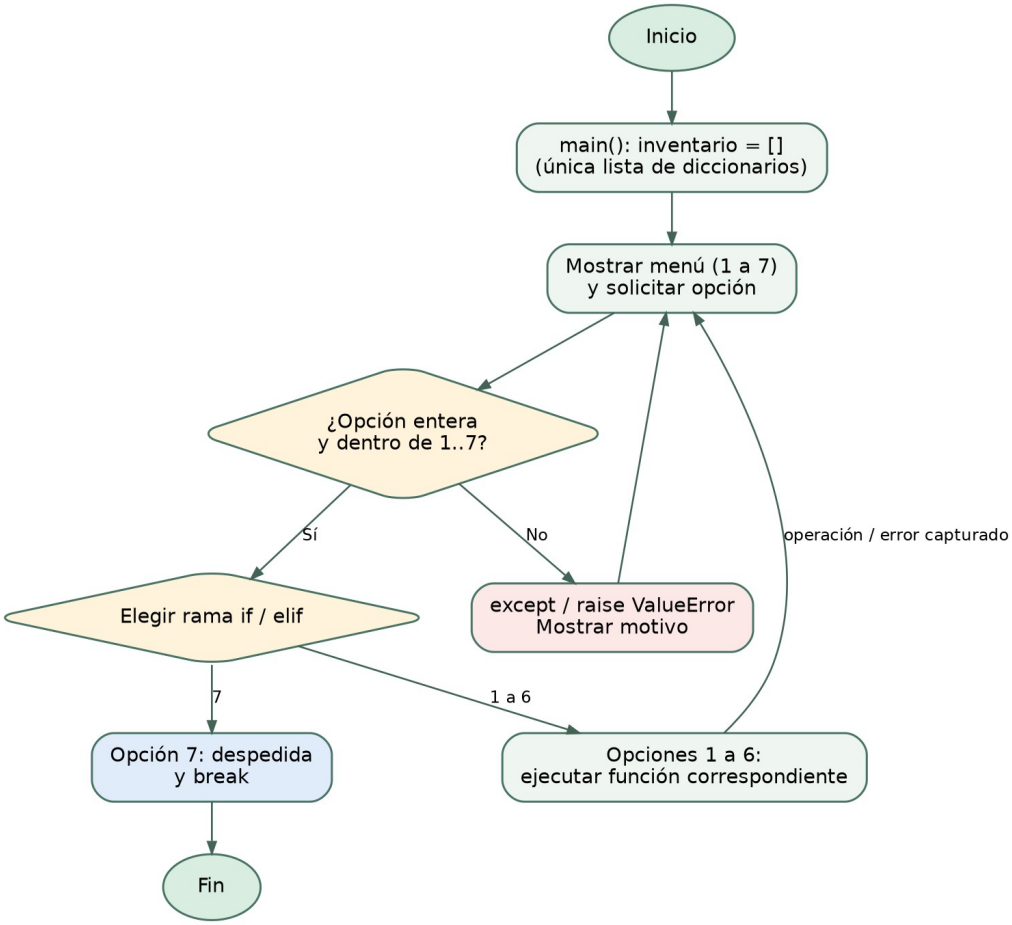
El uso de Python y Visual Studio Code se investigó a partir de documentación oficial, citada al final del informe.

8. Diagramas de flujo de la solución

Se presentan cuatro diagramas construidos a partir de las decisiones reales del archivo Python. Incluyen el menú persistente, los reintentos de carga, el alta individual y los movimientos de existencias. Las flechas identifican el retorno al menú y los puntos en los que se rechaza una operación.

Figura 1. Flujo general del menú principal

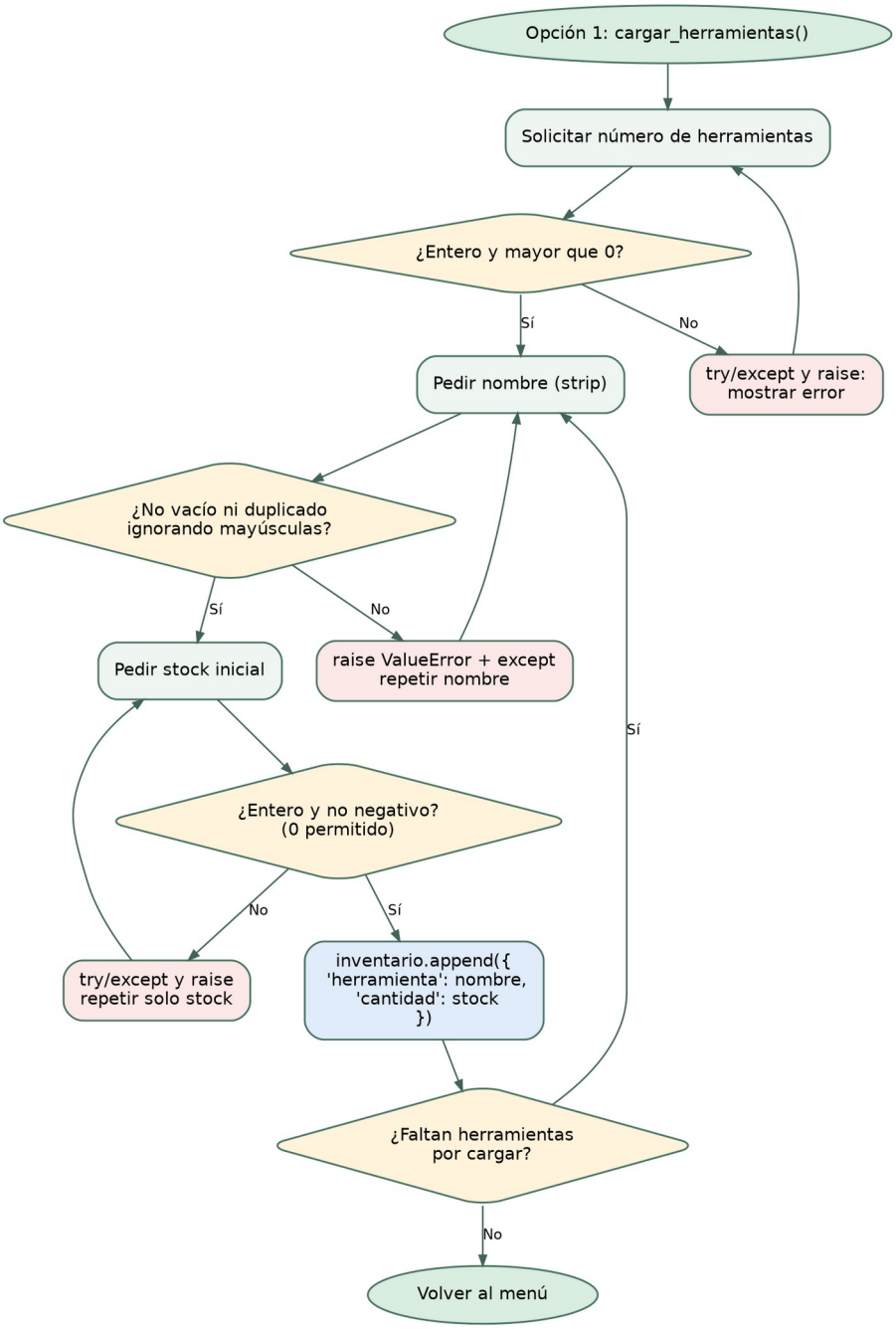
El menú comienza con una única lista inventario dentro de main(). Las opciones válidas ejecutan una función y luego regresan al menú. Una opción inválida muestra el motivo y no finaliza el programa.



Fuente: elaboración propia a partir del archivo ferreteria_nuevo.py.

Figura 2. Flujo de la carga múltiple y validación de datos

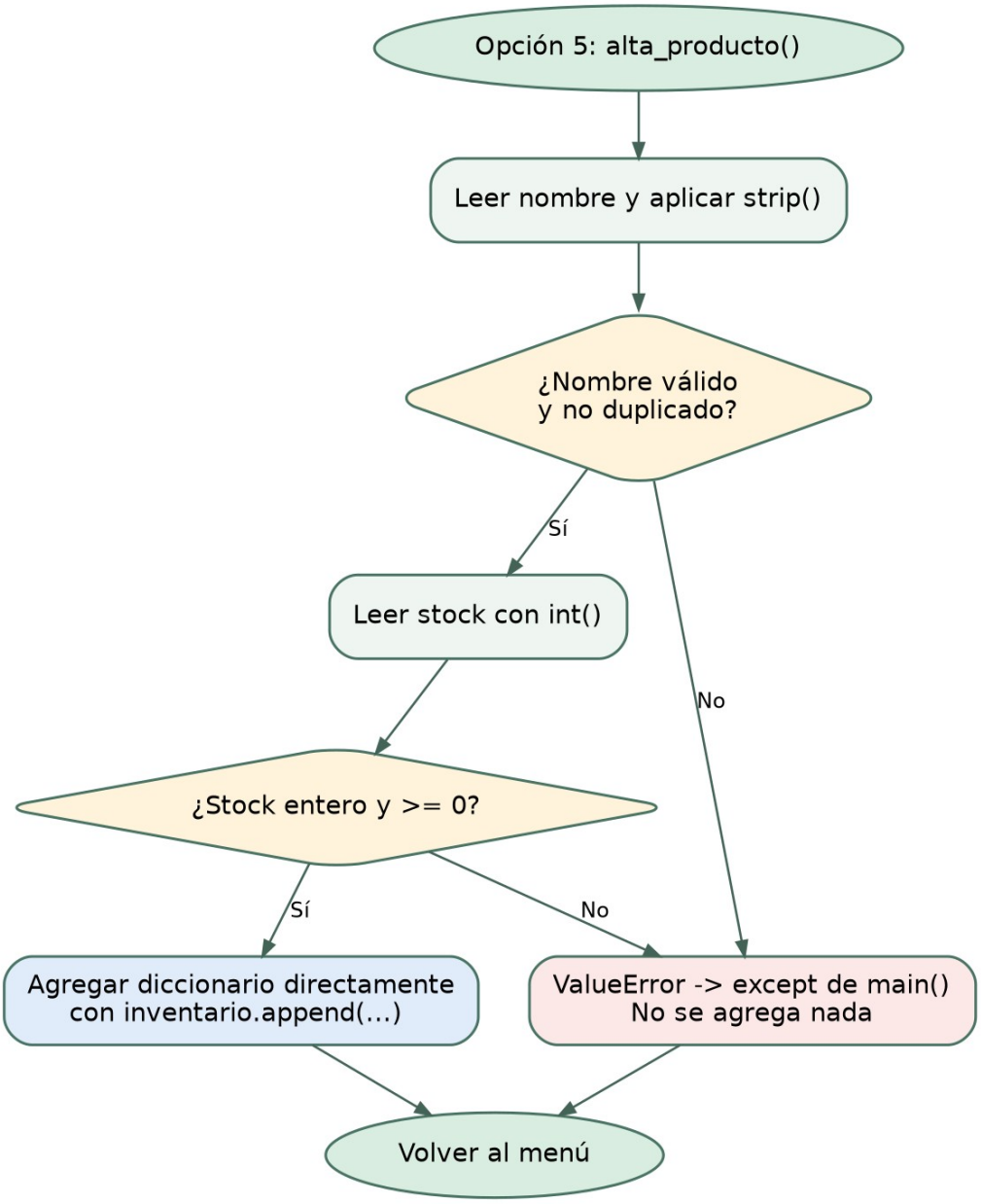
En esta opción se comprueban por separado la cantidad de herramientas, el nombre y el stock inicial. Los errores provocan reintentos locales; el diccionario solo se agrega al inventario cuando sus dos datos son válidos.



Fuente: elaboración propia a partir del archivo ferreteria_nuevo.py.

Figura 3. Flujo del alta individual de productos

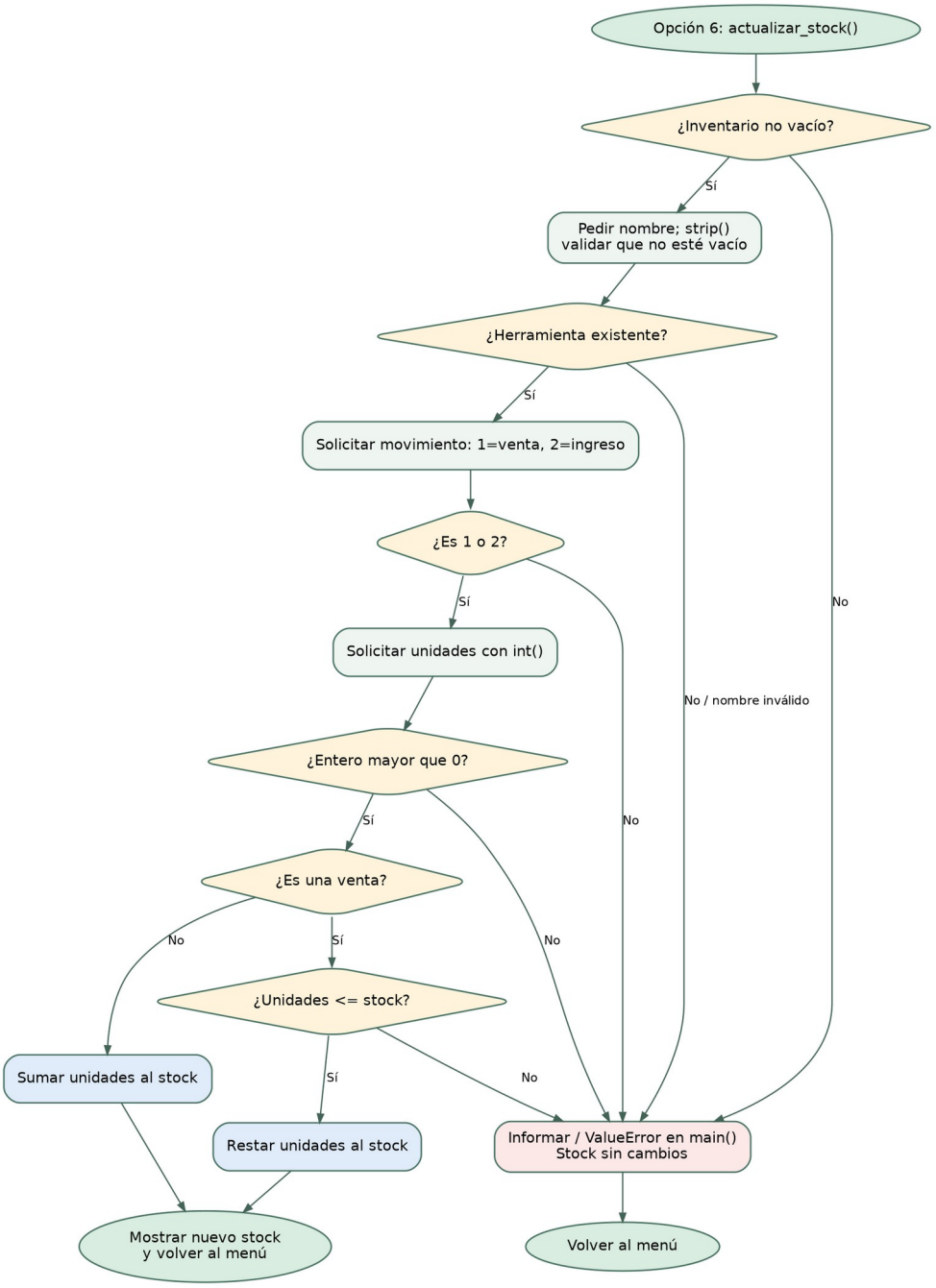
A diferencia de la carga múltiple, el alta individual no repite los datos incorrectos dentro de su función. Si se produce un ValueError, main() informa el problema y vuelve a ofrecer el menú.



Fuente: elaboración propia a partir del archivo ferreteria_nuevo.py.

Figura 4. Flujo de venta e ingreso de mercadería

La actualización verifica los datos en orden. Si falla una condición, la función termina sin modificar existencias. Solo al finalizar correctamente las validaciones se ejecuta una suma o resta del stock.



Fuente: elaboración propia a partir del archivo ferreteria_nuevo.py.

9. Pruebas de funcionamiento

Se verificaron los casos esenciales de uso en Visual Studio Code. Además, se ejecutaron siete grupos de pruebas automatizadas sobre el archivo ferreteria_nuevo.py, utilizando entradas simuladas y revisando los mensajes producidos. Los siete grupos dieron el resultado esperado; estas pruebas técnicas no sustituyen la demostración personal exigida para el video.

9.1. Casos positivos observados en la ejecución de la aplicación

Caso	Entrada / estado	Resultado observado
Carga inicial	Martillo 10; Pala 0	Se registran ambos productos.
Visualización	Opción 2	Se muestran nombres y stocks.
Consulta	Opción 3: Martillo	Se informa el stock disponible.
Agotados	Pala con stock 0	Se muestra Pala como agotada.
Alta individual	Destornillador con 15	Se incorpora al inventario.
Venta	Martillo 10; venta de 3	El stock queda en 7.
Ingreso	Pala 0; ingreso de 5	El stock queda en 5.
Reporte sin agotados	Todos los stocks son positivos	Se informa que no hay agotados.

9.2. Errores y situaciones límite revisadas

Prueba	Comportamiento esperado y obtenido
Nombre repetido en alta	Se rechaza MARTILLO si ya existe Martillo.
Venta de 20 con 7 disponibles	Se rechaza la venta y el stock sigue en 7.
Menú abc o fuera de rango	Se muestra un error y se conserva el menú.
Inventario vacío	Opciones de consulta/visualización informan que no hay productos.
Cantidad de carga inválida	Se reintenta 0, -1, abc o 3.5 dentro de la opción 1.
Nombre en blanco o de solo espacios	Se informa el error sin registrar herramientas incompletas.
Stock -1 o no entero	No se admite y se solicita corrección según la opción.
Consulta sin coincidencia	Se informa que la herramienta no pertenece al catálogo.
Movimiento inválido	No se realiza ninguna actualización de stock.
Unidades de movimiento no válidas	Se rechazan 0, negativas y entradas no enteras.
Venta igual al stock	La herramienta queda con stock 0 y aparece entre agotados.
Ingreso desde stock 0	La herramienta vuelve a tener existencias.

Nombre con espacios o mayúsculas distintas	Se encuentra la herramienta (opciones 3 y 6) o se rechaza como duplicado (opciones 1 y 5), por ejemplo con MARTILLO si ya existe Martillo.
Alta con stock 0	Se acepta, el producto se agrega al final del inventario y aparece en el reporte de agotados.
Stock inválido en una carga múltiple	Se vuelve a pedir solo el stock conservando el nombre; al final se agregan exactamente las herramientas indicadas.
Segunda ejecución de la carga	Se conservan las herramientas anteriores, se agregan las nuevas y se controlan los duplicados contra todo el inventario.

Los resultados reflejan la verificación del código suministrado. Para la demostración audiovisual se seleccionarán ejemplos de éxito y error en el orden de las opciones del menú, tal como exige la cátedra.

9.3. Evidencias de ejecución en Visual Studio Code

Las siguientes capturas documentan pruebas manuales realizadas en la terminal del programa. Se seleccionaron casos que permiten observar validaciones, control de existencias y movimientos de inventario.

Figura 5. Manejo de errores en la cantidad de herramientas.

Se ingresó una cantidad negativa (-3) y un valor no numérico (abc). El sistema informó ambos errores y volvió a solicitar el dato, permitiendo continuar con una cantidad válida de dos herramientas.



Fuente: captura propia de la ejecución del sistema de inventario.

Figura 6. Validación del stock inicial.

Se intentó registrar un Martillo con stock -5 y el sistema rechazó la cantidad negativa. Luego aceptó el valor 10. También se registró un Taladro con stock inicial 0, válido para representar una herramienta agotada.

```
Seleccione una opción: 1

--- CARGA DE HERRAMIENTAS ---
¿Cuántas herramientas desea cargar?: 2
Nombre de la herramienta 1: Martillo
Stock inicial: -5
Error: El stock inicial no puede ser negativo.
Stock inicial: 10
Herramienta registrada correctamente.
Nombre de la herramienta 2: Taladro
Stock inicial: 0
Herramienta registrada correctamente.
```

Fuente: captura propia de la ejecución del sistema de inventario.

Figura 7. Reporte de herramientas agotadas.

Con el Taladro cargado con 15 unidades (en una ejecución independiente de la Figura 6), se registró la venta de 15 unidades y su stock pasó a cero. Al consultar el reporte de agotados, el sistema identificó correctamente la herramienta sin existencias.

```
7. Salir
Seleccione una opción: 6

--- ACTUALIZACIÓN DE STOCK ---
Nombre de la herramienta: Taladro
1. Venta (restar unidades)
2. Ingreso (sumar unidades)
Tipo de movimiento (1 o 2): 1
Cantidad de unidades: 15
Venta registrada correctamente.
Nuevo stock de taladro: 0

===== SISTEMA DE CONTROL DE INVENTARIO =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 4

--- REPORTE DE AGOTADOS ---
Herramienta agotada: taladro

===== SISTEMA DE CONTROL DE INVENTARIO =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción:
Enter command number:
```

Fuente: captura propia de la ejecución del sistema de inventario.

Figura 8. Ingreso de mercadería y consulta de stock.

Se ingresaron 15 unidades del Taladro, cuyo stock era cero. La aplicación confirmó el movimiento y la consulta posterior mostró 15 unidades disponibles.

```
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 6

--- ACTUALIZACIÓN DE STOCK ---
Nombre de la herramienta: Taladro
1. Venta (restar unidades)
2. Ingreso (sumar unidades)
Tipo de movimiento (1 o 2): 2
Cantidad de unidades: 15
Ingreso de mercadería registrado correctamente.
Nuevo stock de taladro: 15

===== SISTEMA DE CONTROL DE INVENTARIO =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 3

--- CONSULTA DE STOCK ---
Nombre de la herramienta: Taladro
Stock disponible de taladro: 15
```

Fuente: captura propia de la ejecución del sistema de inventario.

10. Resultados, límites y conclusiones

El sistema desarrollado permite cargar herramientas, consultar existencias, identificar agotados, dar de alta nuevos productos y registrar entradas y salidas de mercadería. Las comprobaciones realizadas indican que las principales operaciones funcionan y que los casos de error ensayados no alteran indebidamente el inventario.

La parte más importante del diseño fue separar las funciones y decidir en qué momento modificar el stock. Primero se validan los datos y recién después se guarda una herramienta o se actualizan las unidades. De esta manera se evita que una operación incorrecta deje información a medias.

También fue útil distinguir la carga múltiple del alta individual: en la primera, el usuario puede corregir el dato dentro de la misma operación; en la segunda, un error lo devuelve al menú. Esta diferencia permitió practicar bucles, condiciones y excepciones de manera combinada.

Como límite principal, el inventario existe solamente en memoria. Si se cierra el programa, sus datos no se conservan. En una versión futura se podría estudiar la persistencia, pero no se

incorporó aquí para cumplir las restricciones de la consigna. El trabajo permitió relacionar estructuras de datos, modularización y validación de entradas en un programa concreto.

11. Fuentes consultadas

- Universidad Tecnológica Nacional. TPI - Programación 1: Sistema de Control de Inventario. Consigna de la cátedra (21 páginas, 2026).
- Python Software Foundation. Tutorial oficial de Python - Estructuras de datos. https://docs.python.org/es/3/tutorial/datastructures.html
- Python Software Foundation. Tutorial oficial de Python - Errores y excepciones. https://docs.python.org/es/3/tutorial/errors.html
- Microsoft. Getting Started with Python in VS Code. https://code.visualstudio.com/docs/python/python-tutorial
- Archivo fuente del trabajo: ferreteria_nuevo.py, incluido en el ZIP de entrega.